Trabalho Prático CDN

Diogo Pereira (8200594) · Duarte Sampaio (8190553) · Hugo Guimarães (8220337)

14/12/2025

1 Justificação da Arquitetura Adotada

Na Figura 1 está representada a arquitetura da aplicação desenvolvida no AWS, e como é possível visualizar, ela encontra-se devidida nas duas principais camadas: o frontend e o backend.

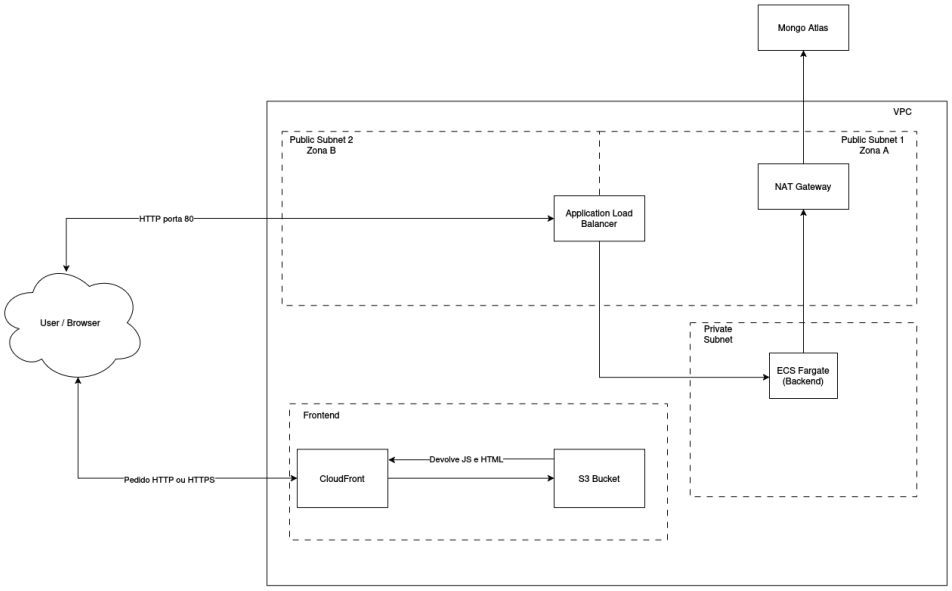


Figura 1: Diagrama de Arquitetura

- **Frontend:** Este está num *bucket* S3 privado e acessível via Amazon CloudFront (CDN).
- **Backend:** Este está implementado numa VPC (*Virtual Private Cloud*) segmentada. A API executa sobre Amazon ECS com Fargate, estando numa subnet privada de modo a estar totalmente isolada da internet. O tráfego de entrada é gerido por um *Application Load Balancer* (ALB) localizado nas duas subnets públicas, e está responsável por encaminhar os pedidos para os containers.

O backend comunica com uma base de dados MongoDB, mas esta não existe dentro do AWS, mas sim como um serviço externo, o MongoDB Atlas, portanto o URI de conexão ao mesmo é passado como uma variável de ambiente na configuração do Open Tofu. Estando depois disponível à API através do intermédio da NAT Gateway, que é necessário visto que a API encontra-se numa subnet privada.

2 Justificação das Decisões Tomadas

2.1 Porque se usou o ECS Fargate

Utilizou-se ECS Fargate porque este é mais simples do que configurar manualmente os servidores em EC2, para além deste ser serverless, debitando somente o que gasta, adequando-se melhor às necessidades do

grupo que estão limitados aos créditos que a Amazon oferece.

2.2 Integração Dinâmica Frontend-Backend

De modo a facilitar o processo de deploy, fez-se um ficheiro *config.js* no forntend que tem credenciais de acesso à API, mas como o endereço do mesmo só é conhecido após o deploy, então utilizou-se o *aws_s3_object* que injeta dinamicamente um ficheiro *config.js* no bucket S3 contendo o endpoint do ALB.

2.3 Porque se utilizou o MongoDB Atlas

No começo tentou-se seguir a sugestão do professor de usar o DynamoDB, mas como havia pouco tempo para a realização do trabalho, e não se estava a ter sucesso na implementação do mesmo na API, então decidiu-se utilizar o MongoDB visto que já foi utilizado diversas vezes em outras unidades curriculares. No entanto, devido à falta de tempo, e ao receio de acabarem os créditos do AWS, decidiu-se utilizar o MongoDB Atlas, e caso houvesse tempo, o que não aconteceu, implementar-se-ia a base de dados diretamente no AWS.

3 Como Executar a Aplicação

Como o relatório só pode ter até 3 páginas, então foi colocado na raiz do projeto um ficheiro markdown denominado HOW_TO_RUN.md.

4 Divisão do Trabalho pelo Grupo

- Diogo Pereira:
  - Frontend local
  - Backend local
  - Open Tofu do S3 + CloudFront
- Duarte Sampaio:
  - Diagrama de Arquitetura
  - Configuração inicial do ECS Fargate da API
  - VPC e Subnets
- Hugo Guimarães:
  - Open Tofu do Load Balancer
  - Criação do repositório ECW
  - Adaptação do ECS para Load Balncer

5 Utilização de Inteligência Artificial

Para o desenvolvimento deste projeto foram utilizadas algumas inteligências artificiais como auxílio ao desenvolvimento do mesmo. No entanto a utilização das mesmas não foi utilizada como substituto do trabalho necessário para a aquisição das competências necessárias para o desenvolver, sendo somente utilizadas como recurso para facilitar trabalho repetitivo, demorado e não relevante para o principal objetivo do projeto. Tendo sido utilizadas para as seguintes tarefas:

- Atribuição das classes bootstrap no frontend para ficar mais esteticamente agradável;
- Escrita da role do IAM para o ECS poder escrever logs e fazer pull de imagens docker;
- Resolução de problemas de execução, sobretudo quando o erro não era explícito (Ex: Esclareceu o que era e como implementar o CloudWatch para poder fazer Troubleshooting);
- Esclarecimento de dúvidas sobre as funcionalidades do AWS, e como estas se integram umas com as outras

6 Referências para a construção do trabalho

- **Criar um bucket S3 em OpenTofu:** https://medium.com/@2017tejasgupta/created-your-first-s3-bucket-with-opentofu-7c806bbed218
- **Criar um bucket S3 em Terraform e expor com o CloudFront:** https://medium.com/@ssare/make-s3-bucket-public-with-object-read-only-via-terraform-0b71393b963e
- **Tabelas de Roteamento:** https://search.opentofu.org/provider/opentofu/aws/latest/docs/resources/route_table
- **Configuração de ECS Fargate:**
  - https://medium.com/@olayinkasamuel44/using-terraform-and-fargate-to-create-amazons-ecs-e3308c1b9166
  - https://spacelift.io/blog/terraform-ecs https://www.gyden.io/en/content-hub/how-to-setup-amazon-ecs-fargate-using-terraform
  - https://search.opentofu.org/module/cloudposse/ecs-cluster/aws/latest
  - https://search.opentofu.org/provider/opentofu/aws/latest/docs/resources/ecs_service
  - https://spacelift.io/blog/terraform-ecs
- **Configuração de VPC e Subnets:**
  - https://search.opentofu.org/provider/opentofu/aws/latest/docs/resources/vpc
  - https://dev.to/vinod827/creating-custom-vpc-on-aws-using-opentofu-3760
  - https://search.opentofu.org/provider/opentofu/aws/latest/docs/resources/subnet
  - https://dev.to/vinod827/creating-custom-vpc-on-aws-using-opentofu-3760